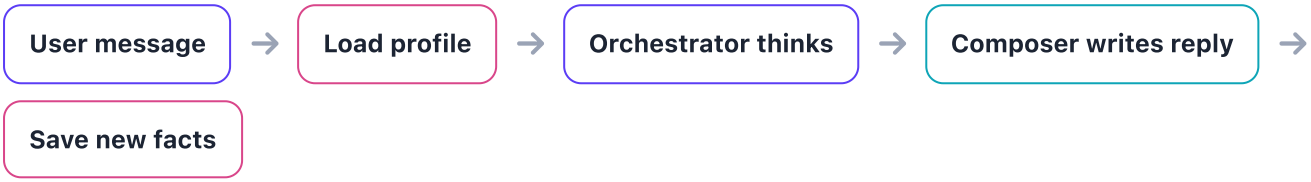
BRAINCC · PLAIN-ENGLISH WALKTHROUGH

Long-Term Memory, Step by Step

How the system remembers a shopper across conversations — and feeds that memory into the Orchestrator and Composer. Read it top to bottom.

The whole thing in one line



The shopper's profile lives in a small separate service called `braincc-memory`. The Orchestrator reads it *before* answering and updates it *after* answering. Everything is keyed by one stable id: `user_token`.

PHASE A — BEFORE THE ANSWER (READ)

1 A message arrives LIVE

The shopper sends a chat message. It comes with a `user_token` (who they are, same every visit) and a `context_id` (this conversation).

2 Orchestrator asks: "Do we know this shopper?" READ

It calls `apply_persona(user_token)`. If the profile is already cached, it gets back a short `<user_profile>` summary (gender, sizes, style…). If not, it fetches it from `braincc-memory` in the background for next time.

3 The profile is glued onto the prompt READ

That profile summary is added to the Orchestrator's system prompt. Now the model "knows" the shopper before it even reads their message.

4 The Orchestrator (router) decides what to do

It reads the message + profile and picks a specialist — e.g. recommendations, cart, product, or image search — and hands off the task.

Note: the specialist sub-agents do *not* get the profile — only the Orchestrator and Composer do. Sub-agents just receive the task and the `context_id`.

PHASE A CONTINUED — WRITING THE REPLY

5 The Composer writes the final reply READ

After the specialist returns its result, the **Composer** turns it into a natural reply. It calls `apply_persona(user_token)` again (same cache) so the wording matches what we know about the shopper.

6 The answer streams back to the shopper LIVE

The reply is sent out over SSE. From the shopper's point of view, the turn is now done.

PHASE B — AFTER THE ANSWER (WRITE, HAPPENS IN THE BACKGROUND)

7 Grab the full conversation WRITE

The Orchestrator pulls the complete turn transcript from its MongoDB checkpointer (keyed by `context_id`).

8 Send it to memory to "reflect" WRITE

`record_turn(user_token, messages)` fires off to `braincc-memory`. This is fire-and-forget — it does not slow down the reply.

9 Memory extracts new facts with an LLM WRITE

In the background, `braincc-memory` reads the conversation and pulls out durable facts ("likes minimalist style", "size M"). It merges them into the stored `ShopperProfile`.

10 The cache is refreshed for next time

The updated profile is pulled back into the Orchestrator's cache, so the *next* message already has the latest picture of the shopper. Loop back to Step 1.

Key idea: memory is **read before** the model answers (to personalize) and **written after** the model answers (to learn). The Orchestrator and Composer share the exact same cached profile — there's no extra service call between them.

Where the profile lives: a separate service `braincc-memory` (port 9100), keyed by `user_token`. It's structured facts + an LLM — not a vector database. In the current version it's held in memory, so it resets if that service restarts.

What a user profile looks like

The stored memory is a **multi-subject shopping profile**. One shopper (`user_id` = their `user_token`) can hold many **subjects** — themselves *and* other people they shop for (kids, partner, gifts). Each subject carries identity, sizes, and preferences.

TOP LEVEL · SHOPPERPROFILE

FIELD	TYPE	MEANING
`user_id`	`str`	Stable owner id (the shopper's `user_token`)
`updated_at`	`datetime`	Last time the extractor wrote to the profile
`subjects`	`dict[str, Subject]`	Map of `subject_id` → subject. `"self"` = the shopper

EACH SUBJECT · SUBJECTPROFILE

FIELD	TYPE	MEANING
`subject_id`	`str`	`"self"`, or a slug like `"sister_anika"`
`subject_role`	`self | other`	Is this the shopper, or someone else?
`subject_label`	`str`	Human label — "myself", "my sister Anika"
`relationship`	`str | null`	sibling / parent / spouse / child / friend (null for self)
`age`	`int | null`	Age in years, 0–120, if stated
`gender`	`male | female | non_binary | unspecified | null`	Strict set; `unspecified` = named but no gender; `null` = not modelled
`topwear_size`	`str | null`	Shirt / top size — "S", "M", "18"
`bottomwear_size`	`str | null`	Pant / trouser size — "32W", "38"
`footwear_size`	`str | null`	Shoe size — "UK 9", "42"
`style_preferences`	`list[str]`	minimal, streetwear, scandinavian…
`color_preferences`	`list[str]`	Colours / palettes spoken of positively
`occasions`	`list[str]`	work, wedding, gym…
`avoid_materials`	`list[str]`	Fabrics / materials to avoid
`updated_at`	`datetime`	Last change to this subject

Why subjects? A shopper might say "I need running shoes, and a birthday gift for my sister Anika." That creates two subjects — `self` and `sister_anika` — so the assistant never mixes up their sizes or gender.

A full profile — stored vs. injected

Left: how a complete profile is **stored** in the memory service (JSON). Right: the exact `<user_profile>` block that gets **injected** into the Orchestrator & Composer prompt at Step 2/5.

STORED — ShopperProfile (JSON)

```
// key = user_token
{
  "user_id": "usr_8c1f...e29",
  "updated_at": "2026-06-02T13:31Z",
  "subjects": {
    "self": {
      "subject_id": "self",
      "subject_role": "self",
      "subject_label": "myself",
      "relationship": null,
      "age": 34,
      "gender": "female",
      "topwear_size": "M",
      "bottomwear_size": "30W",
      "footwear_size": "UK 6",
      "style_preferences": ["minimal",
                            "scandinavian"],
      "color_preferences": ["navy", "beige"],
      "occasions": ["work"],
      "avoid_materials": ["wool"]
    },
    "sister_anika": {
      "subject_id": "sister_anika",
      "subject_role": "other",
      "subject_label": "my sister Anika",
      "relationship": "sibling",
      "gender": "female",
      "topwear_size": "S",
      "style_preferences": ["boho"],
      "occasions": ["birthday gift"]
    }
  }
}
```

INJECTED — into the system prompt

```
KNOWN SHOPPER FACTS — verified ground
truth from prior conversations.
RULE 1 — confirm a saved size once
before adding to cart, then apply
silently. RULE 2 — elsewhere, use
facts silently; never re-ask.
<user_profile>
  <subject id="self" role="self"
           label="myself">
    age: 34
    gender: female
    topwear_size: M
    bottomwear_size: 30W
    footwear_size: UK 6
    style_preferences: minimal, scandinavian
    color_preferences: navy, beige
    occasions: work
    avoid_materials: wool
  </subject>
  <subject id="sister_anika" role="other"
           label="my sister Anika"
           relationship="sibling">
    gender: female
    topwear_size: S
    style_preferences: boho
    occasions: birthday gift
  </subject>
</user_profile>
```

How the two relate: the JSON is the source of truth in `braincc-memory`. The renderer drops empty fields, lists `self` first, and wraps everything in the two guard rules — so the model treats sizes as "confirm once before cart, use silently everywhere else."